UNIVERSIDAD AUTÓNOMA JUAN MISAEL SARACHO
FACULTAD DE CIENCIAS DE LA INGENIERÍA DE RECURSOS NATURALES Y TECNOLOGÍAS
CARRERA DE INGENIERÍA INFORMÁTICA

INFORME TÉCNICO FINAL

Sistema RAG Multi-Dominio para la Consulta de Normativa Financiera y Tributaria de Bolivia con Búsqueda Híbrida (Vectorial + Léxica)

Materia	TEL420 – Sistemas Paralelos (IIY) – G1/2026
Docente	Ing. Elías Cassal Baldiviezo
Estudiante	Helen Abigail Soto Renjifo
Registro Universitario	e123181
Cédula de Identidad	10633918
Fecha	Octubre de 2026

Proyecto rag-multidominio — Pipeline RAG con PostgreSQL + pgvector

1. Contexto, problema y objetivos

Contexto. El proyecto aborda la consulta de normativa boliviana de dos dominios: financiero (Banco Central de Bolivia y ASFI) y tributario (Impuestos Nacionales y Aduana). La información oficial es extensa y dispersa, lo que dificulta obtener respuestas trazables.

Problema. Se requiere recuperar evidencia verificable, citando la fuente, con aislamiento estricto por dominio y tolerancia a los límites de la API de Gemini (429 / RESOURCE_EXHAUSTED / cuota agotada), sin inventar datos.

Objetivo general. Construir un sistema RAG multi-dominio con aislamiento estricto entre colecciones y presentación de fuentes verificables.

Objetivos específicos: ingesta web concurrente y troceado en fragmentos solapados; embeddings de 768 dimensiones en PostgreSQL + pgvector; búsqueda vectorial (conservada) y búsqueda léxica nativa de PostgreSQL; recuperación híbrida mediante Reciprocal Rank Fusion (RRF); fallback léxico si falla el embedding; y chat web con fuentes y pruebas automatizadas.

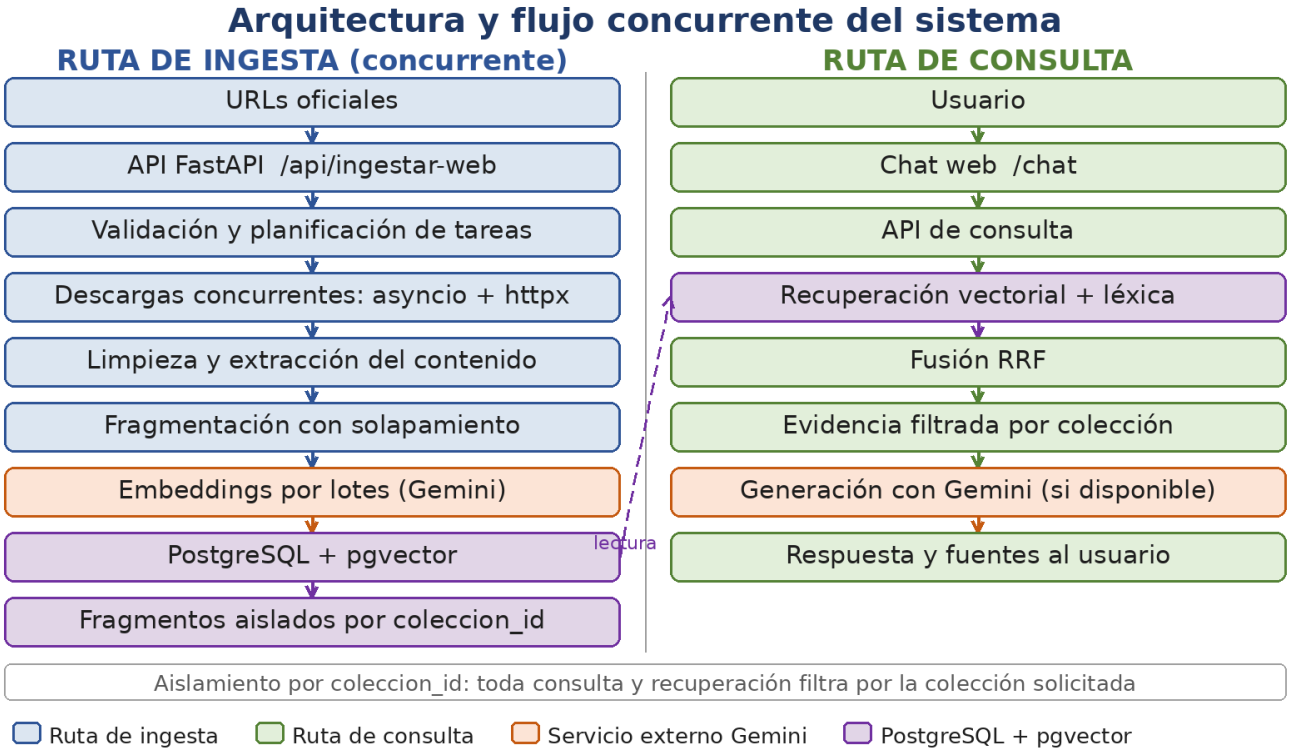


Figura 1. Arquitectura y flujo concurrente del sistema.

2. Arquitectura técnica y aislamiento

2.1 Servicios y almacenamiento

Backend FastAPI (Python 3.12); PostgreSQL 16 + pgvector 0.8.7; embeddings gemini-embedding-001 (768 dimensiones); generación con Gemini a temperatura 0.0; despliegue con Docker Compose. La tabla normativa_bancaria(id, coleccion_id, contenido, embedding vector(768), origen_url, created_at) tiene un índice por coleccion_id.

2.2 Aislamiento por coleccion_id

La columna coleccion_id es obligatoria (CHECK no vacío) e indexada. La recuperación vectorial exige la colección como primer parámetro y el servicio verifica que toda evidencia pertenezca a la colección pedida antes de invocar al LLM (si no, ErrorContaminacion). El aislamiento se aplica tanto a la búsqueda vectorial como a la léxica.

2.3 Función SQL/RPC match_normativa_coleccion

Resuelve la búsqueda vectorial por similitud dentro de una sola colección. Es una función LANGUAGE sql STABLE que recibe los parámetros de la tabla y devuelve evidencia:

Parámetro	Tipo / valor	Función
p_coleccion_id	varchar (1.º, obligatorio)	Aísla: filtra nb.coleccion_id = p_coleccion_id
p_query_embedding	vector (768)	Embedding de la consulta
p_match_threshold	double, por defecto 0.5	Similitud mínima aceptada
p_match_count	integer, por defecto 10	Límite de resultados (top-k)

Usa pgvector con el operador coseno <=> (distancia), y expone similitud = 1 − distancia. El umbral p_match_threshold descarta evidencia poco relevante y p_match_count acota el número de filas. El filtro obligatorio por p_coleccion_id evita la contaminación entre colecciones: la RPC nunca devuelve filas de otra colección.

2.4 Recuperación vectorial, léxica e híbrida

Vectorial: la RPC anterior, sobre pgvector y distancia coseno (recuperación semántica).

Léxica: texto completo nativo de PostgreSQL (diccionario spanish, sin pg_trgm ni unaccent). Extrae los lexemas de la consulta y exige intersección ≥ 1 o coincidencia de frase exacta (resoluciones/artículos), con orden reproducible. Complementa a la vectorial, no la sustituye.

Híbrida: fusión por RRF (Σ 1/(k+rank), k = 60) con bono por coincidencia exacta y deduplicación por id. La similitud solo se conserva si el fragmento provino de la búsqueda vectorial; para resultados solo-léxicos queda en None.

3. Pruebas y benchmark

3.1 Pruebas automatizadas (FASE 9)

347 aprobadas, 0 fallidas y 0 omitidas (1 advertencia no relacionada). Ejecutadas offline con pytest; las de aislamiento y búsqueda léxica operan contra PostgreSQL en solo lectura.

3.2 Benchmark de concurrencia (scraping)

10 URLs oficiales, 3 repeticiones por nivel y carga idéntica (hash b5053f6c8…b20d). Mide solo la fase de scraping (descarga y extracción de contenido), sin embeddings ni inserciones. Fórmulas: S(p) = T(1)/T(p) y E(p) = S(p)/p, con T(1) = 6.931249 s.

Trabajadores	Tiempo medio (s)	Speedup S(p)	Eficiencia E(p)
1	6.931249	1.0000	1.0000
2	3.040423	2.2797	1.1398
4	2.198843	3.1522	0.7881
8	1.773352	3.9086	0.4886

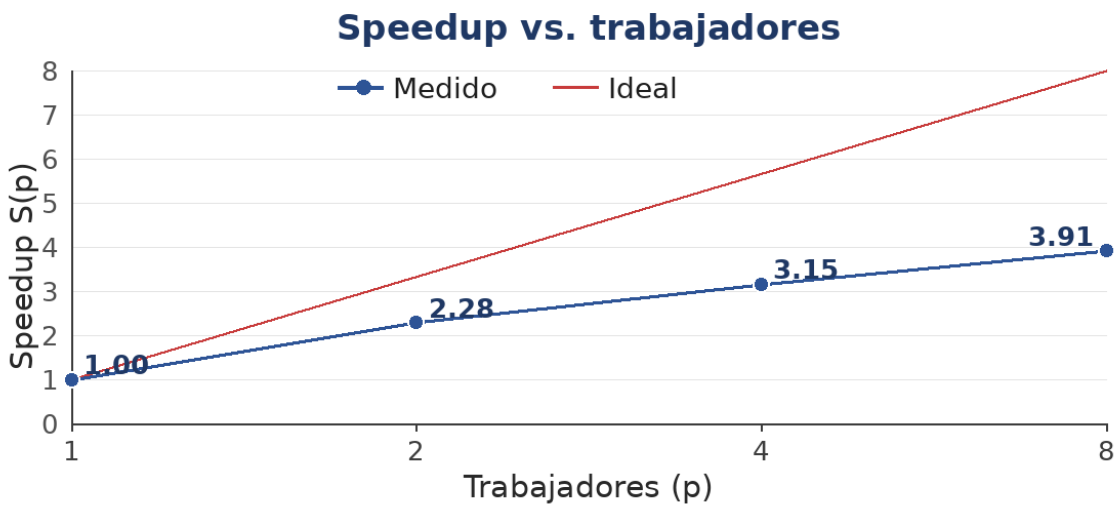


Figura 2. Speedup frente al número de trabajadores (con ideal S(p) = p).

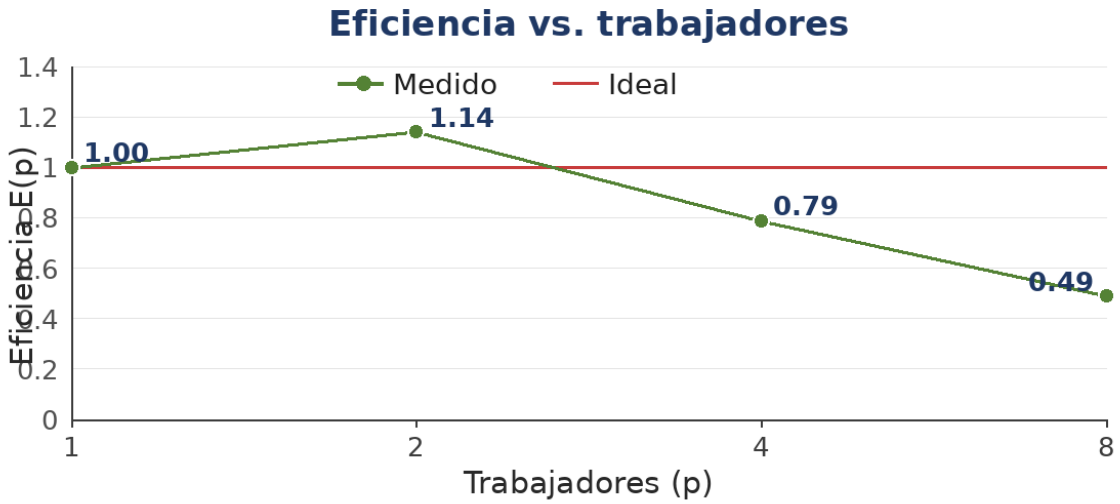


Figura 3. Eficiencia frente al número de trabajadores (con ideal E(p) = 1).

4. Interpretación y escalabilidad

El scraping concurrente redujo el tiempo y alcanzó un speedup de 3.91× con 8 trabajadores. La eficiencia con 2 trabajadores es mayor que 1 (1.1398): un resultado aparentemente superlineal que se explica por el solapamiento de las esperas de red y la variabilidad de las mediciones, por lo que debe interpretarse con cautela y no como prueba concluyente. A partir de 4 trabajadores la eficiencia disminuye, coherente con los límites del recurso paralelizable.

Escalabilidad y límites de API de terceros. La concurrencia solapa las esperas de red y reduce el tiempo en cargas apropiadas, pero más trabajadores no garantizan una aceleración proporcional. Los embeddings dependen de los límites, la cuota y la disponibilidad del proveedor externo: los errores 429 / RESOURCE_EXHAUSTED pueden impedir que la ingesta integral termine aunque el scraping sea más rápido, y los 5xx provocan reintentos que aumentan la latencia. El batching, la concurrencia limitada, el backoff y los reintentos finitos ayudan a gestionar estos límites, pero no eliminan las cuotas. Conforme a la ley de Amdahl, la parte no paralelizable y el cuello de botella más restrictivo (red, proveedor de embeddings, base de datos) limitan el speedup total. Por ello no se afirma un speedup integral de ingesta.

5. Conclusiones y limitaciones

Conclusiones. El sistema integra ingesta concurrente, aislamiento estricto por coleccion_id y búsqueda híbrida; la fusión RRF mejora la recuperación de identificadores sin renunciar a la semántica; ante fallo del embedding se continúa con la búsqueda léxica; y el chat degrada a solo-recuperación si la generación responde 503.

Limitaciones. Las 347 pruebas aprobadas corresponden a la FASE 9; las pruebas offline no demuestran por sí solas que toda respuesta real del chat sea correcta. La generación real con Gemini y la búsqueda vectorial de preguntas nuevas no quedaron validadas en la FASE 9 por los problemas de cuota. La colección tributaria tiene cobertura insuficiente para responder con evidencia sobre IVA, y el benchmark integral quedó limitado por los errores 429. El benchmark de la Figura 2 mide solo el scraping y no los embeddings ni las inserciones.